Regime Belief Probabilities for Belief-Aware RL: Design, Failure Modes, and the MLP+Kalman Solution

Belief-Aware RL Project — Design Report

Belief-Aware RL Project

May 2026

Contents

1 Motivation: Why Regime Probabilities? 1
1.1 The Two Regimes . . . 1
1.2 Observation Layout . . . 2

2 Step 1: The Oracle (E1 Upper Bound) 2

3 Step 2: The Kim Filter Attempt (and Why It Failed) 2
3.1 The Kim (1994) Regime-Switching Kalman Filter . . . 2
3.2 Why the Kim Filter Cannot Distinguish Val from Mom . . . 3

4 Step 3: The Learned Classifier Approach 3
4.1 Framing as Supervised Classification . . . 3
4.2 Feature Engineering . . . 3
4.3 Sliding Window Input . . . 4
4.4 Dataset Construction . . . 4

5 Step 4: LSTM vs MLP 4
5.1 LSTMRegimeClassifier . . . 4
5.2 MLPRegimeClassifier . . . 4
5.3 Results (20 train / 10 eval episodes) . . . 5

6 Step 5: Adding Kalman Belief Features 5
6.1 Kalman Features . . . 5
6.2 Why These Features Might Help . . . 5
6.3 Empirical Result . . . 5

7 Step 6: Full Architecture Overview 6
7.1 RegimeRandomizedGymnasiumEnv . . . 6

8 Implementation Notes 6
8.1 Bug Fixed: Observation Space Double-Counting . . . 6
8.2 Checkpoint Format . . . 7
8.3 CRITICAL_ENV_BUG: reset() does not clear marked-to-market . . . 7

9 Experimental Design for E3 **7**
9.1 Experiment Variants . . . 7
9.2 Training Protocol . . . 8
9.3 Hypothesis . . . 8
9.4 Ablation Plan . . . 8

10 Saved Artifacts **8**

11 Summary **9**

1 Motivation: Why Regime Probabilities?

The v5 oracle experiment establishes that a PPO agent given the true regime label (Val or Mom) as part of its observation achieves a strong performance upper bound. The question addressed in this document is:

> *Can a PPO agent achieve comparable performance when the regime label is replaced by a learned, per-step probability vector* $\hat{b}_t = (p_{val,t}, p_{mom,t})$ *estimated from observable market data?*

This is the **E3 experiment** in the v5 framework. The motivation is practical: in real trading the regime is never observed directly. An agent that relies on soft, continuously updated regime probabilities generalises beyond the oracle setting without requiring the oracle.

1.1 The Two Regimes

All experiments use two market configurations derived from the v4 sweep:

Regime	**num_momentum_agents**	**num_value_agents**
Val (value-dominated)	0	102
Mom (AAPL calibration)	12	102

In Val episodes the order flow is dominated by Bayesian value agents who mean-revert toward the fundamental. Prices exhibit strong mean-reversion. In Mom episodes twelve additional momentum agents follow moving-average crossover signals, injecting trend-following pressure that partially overwhelms mean-reversion. The optimal policy differs between these two regimes: a value/mean-reversion action is profitable in Val but harmful in Mom.

1.2 Observation Layout

The base daily-investor environment produces a 7-dimensional observation at each 60-second step:

$$o_t = \left[\text{holdings},\ \text{imbalance},\ \text{spread},\ \text{direction},\ r_{t-1},\ r_{t-2},\ r_{t-3}\right]$$

After belief augmentation the observation becomes 9-dimensional:

$$\tilde{o}_t = \left[o_t,\ p_{\text{val,t}},\ p_{\text{mom,t}}\right]$$

where $p_{\text{val,t}} + p_{\text{mom,t}} = 1$. The PPO policy π_θ acts on $\tilde{o}_t$ and must learn to condition its action on the belief.

2 Step 1: The Oracle (E1 Upper Bound)

The oracle wrapper produces the hard one-hot belief

$$b_t^* = \begin{cases} (1,\, 0) & \text{if episode regime is Val} \\ (0,\, 1) & \text{if episode regime is Mom} \end{cases}$$

which is constant for the entire episode. This is implemented in `OracleRegimeWrapper`. At `reset()` the wrapper reads the current regime (set by `RegimeRandomizedGymnasiumEnv`) and pins the belief to the corresponding one-hot vector.

The oracle represents the best possible belief: zero uncertainty, perfectly correlated with the environment, available for free. Any learned belief estimator must close the gap between the no-belief baseline and this oracle.

Why not just use the oracle for final training? The oracle requires ground-truth agent counts, which are unavailable in live markets. It serves only as the *upper bound* benchmark (E1). The goal of E3 is to get close to E1 performance with a belief estimated purely from price and order-flow data.

3 Step 2: The Kim Filter Attempt (and Why It Failed)

3.1 The Kim (1994) Regime-Switching Kalman Filter

The first candidate for per-step belief estimation was the Kim (1994) filter, a well-known closed-form algorithm for hidden Markov models with linear-Gaussian emission distributions. The model uses $K = 3$ regimes:

Kim Regime	**Dynamics**	**Process noise**	**Obs. noise**
VALUE	OU (α_{OU}, $\mu = r_{\text{bar}}$)	Q_{base}	R_{base}
MOMENTUM	Random walk ($\alpha = 1$)	$20 \times Q_{\text{base}}$	$2 \times R_{\text{base}}$
NOISE	OU (α_{OU}, $\mu = r_{\text{bar}}$)	$0.3 \times Q_{\text{base}}$	$15 \times R_{\text{base}}$

The rationale was that the Kim VALUE regime tracks mean-reverting fundamentals (matching v5 Val), while MOMENTUM tracks trending prices with amplified noise (matching v5 Mom).

The algorithm runs forward Kalman passes for each regime in parallel, computing $p_k(\text{regime}_k \mid y_{1:t})$ via the Kim (1994) collapse approximation to avoid the exponentially growing mixture.

3.2 Why the Kim Filter Cannot Distinguish Val from Mom

A smoke test was run to compare Kim filter output in Val vs Mom episodes. The result was striking:

	Val episode	**Mom episode**
p_{value} (Kim)	0.95	0.95
p_{momentum} (Kim)	0.04	0.04

The output is essentially identical in both regimes. The root cause is architectural: **the Kim filter is detecting OU dynamics, which are present in both regimes.**

The ABIDES oracle process is governed by an OU SDE in both Val and Mom. The 12 momentum agents in Mom do not change the fundamental process; they add order-flow autocorrelation and price-impact patterns that are invisible to a filter that models only the univariate mid-price sequence. The Kim filter "sees" mean-reverting prices in both cases and assigns high p_{value} in both. The `KimRegimeBeliefWrapper` was built but is kept in the codebase only as a reference.

Key Insight. The signal that distinguishes Val from Mom is not in the *level or volatility of the mid-price* but in the *autocorrelation structure of order flow and short-horizon returns*: momentum agents create serial correlation in imbalance and trend in lagged returns. A filter that models only mid-price dynamics cannot access this signal.

4 Step 3: The Learned Classifier Approach

4.1 Framing as Supervised Classification

Given that the Kim filter is model-misspecified, we reframe the problem as supervised binary classification. We collect rollout data under each regime, label episodes, and train a classifier to predict regime from a sliding window of observable features. At inference time the classifier outputs a soft probability $(p_{\text{val,t}}, p_{\text{mom,t}})$ that is fed to the PPO policy.

4.2 Feature Engineering

Six features are extracted from the 7-dim observation at each timestep:

Index	Feature	Source	Intuition
0	imbalance	$o_t[1]$	Bid-vol fraction; persistent elevation in Mom
1	spread	$o_t[2]/r_{\text{bar}}$	Normalised bid-ask spread
2	direction	$o_t[3]/r_{\text{bar}}$	Mid − last transaction
3	r_{t-1}	$o_t[4]/r_{\text{bar}}$	Most-recent 60s return
4	r_{t-2}	$o_t[5]/r_{\text{bar}}$	2-step lag
5	r_{t-3}	$o_t[6]/r_{\text{bar}}$	3-step lag

The key feature is `imbalance`: momentum agents systematically drive the order book to one side when a trend is in motion. The lagged return triplet $(r_{t-1}, r_{t-2}, r_{t-3})$ exposes sign autocorrelation characteristic of momentum flow. All features are normalized to $[-1, 1]$ or $[0, 1]$ to ensure stable training.

4.3 Sliding Window Input

Both the LSTM and MLP operate on a sliding window of the last seq_len = 20 feature vectors, giving a (20×6) input tensor. This 20-step window spans 20 minutes of simulated trading, long enough to capture imbalance persistence and return autocorrelation, short enough to be responsive to mid-episode regime transitions if they were present (they are not in the current setup, where regime is fixed per episode).

4.4 Dataset Construction

- **Data collection**: Hold-only rollouts (action = 1 every step) are run for N Val episodes and N Mom episodes. The hold strategy was chosen because it produces no confounding from PPO's own order-flow.
- **Sliding windows**: From each T-step episode, windows $[t, t{+}\text{seq_len})$ for $t = 0, \ldots, T{-}\text{seq_len}$ are extracted, giving approximately $T - 20 \approx 361$ windows per episode.
- **Warmup removal**: The first 5 steps of each episode are discarded because the market has not yet reached a stationary state and the features contain initialisation artefacts.
- **Balance**: Val and Mom windows are balanced 50/50 by construction (equal episode counts).

With $N = 20$ training episodes per regime, the dataset contains 14,480 training windows and 7,240 evaluation windows.

5 Step 4: LSTM vs MLP

5.1 LSTMRegimeClassifier

Architecture:

$$\text{LSTM}(\text{input} = d_f,\ \text{hidden} = 32)\ \rightarrow\ \text{Linear}(32, 2)\ \rightarrow\ \text{softmax}$$

where $d_f \in \{6, 8\}$. The LSTM processes the window sequentially and uses the final hidden state for classification. At inference the hidden state is passed across windows to give the model episode-long memory.

Training. Adam, lr= 3×10^{-4}, weight decay 10^{-4}, CosineAnnealingLR over 40 epochs. Gradient clipping at 1.0. Batch size 256.

Why LSTM fails with small data. The LSTM has 5,186–5,442 parameters and requires many episodes to learn meaningful sequential representations. With only 20 training episodes per regime (14,480 windows), the LSTM overfits to episode-level noise rather than learning the structural autocorrelation pattern. It defaults to predicting the majority class (Val): window-level accuracy stays near 51%, and episode-level accuracy is 45–70% depending on the random seed.

5.2 MLPRegimeClassifier

Architecture:

$$\text{Linear}(20d_f,\ 128)\ \rightarrow\ \text{ReLU}\ \rightarrow\ \text{LayerNorm}(128)\ \rightarrow\ \text{Linear}(128,\ 64)\ \rightarrow\ \text{ReLU}\ \rightarrow\ \text{Linear}(64,\ 2)$$

The MLP flattens the $(20 \times d_f)$ window to a $20d_f$-dim vector. It does not model temporal structure explicitly, but the autocorrelation information is encoded in the joint distribution of the 20-step window.

Why MLP succeeds. With 24,130–29,250 parameters and a simpler hypothesis space (no recurrence), the MLP generalises better from the 14,480-window dataset. Window-level training accuracy reaches 70%, and *episode-level* accuracy (majority vote over all windows in an episode) reaches 100% on both Val and Mom test episodes.

Why 100% episode accuracy with only 56% window accuracy? This is the averaging effect. Each episode provides ∼361 windows. Even if a single window has only 56% probability of being correct, the mean p_{val} across all 361 windows concentrates tightly around the true class probability due to the law of large numbers. The episode-level decision (argmax of mean) is then always correct for the regimes tested.

5.3 Results (20 train / 10 eval episodes)

Model	**Overall acc**	$\bar{p}_{\text{val}}$ | Val	$\bar{p}_{\text{mom}}$ | Mom
LSTM (raw)	70%	0.500	0.500
MLP (raw)	100%	0.563	0.582
LSTM (+Kalman)	70%	0.500	0.500
MLP (+Kalman)	100%	**0.594**	**0.611**

6 Step 5: Adding Kalman Belief Features

The base feature set uses only the raw observation vector. A natural question is whether the Kalman belief tracker output — which summarises the agent's posterior over the latent fundamental F_t — provides additional signal for regime discrimination.

6.1 Kalman Features

Two additional features are appended to each timestep's feature vector when `use_kalman=True`:

$$\texttt{belief_dev}_t = \frac{\hat{\mu}_t - \text{mid}_t}{r_{\text{bar}}} \qquad \text{(normalised deviation of posterior mean from mid-price)}$$

$$\texttt{belief_std}_t = \frac{\hat{\sigma}_t}{r_{\text{bar}}} \qquad \text{(normalised posterior uncertainty)}$$

where $\hat{\mu}_t$ and $\hat{\sigma}_t$ are the Kalman posterior mean and standard deviation at step t.

6.2 Why These Features Might Help

In a Val episode the value agents' order flow is closely aligned with the Kalman estimate of the fundamental, so `belief_dev` should be small and `belief_std` should be low. In a Mom episode the momentum agents drive prices away from the fundamental, inflating `belief_dev` and potentially `belief_std` as well. The Kalman tracker thus provides a low-frequency signal of "how far is the price from the model's fundamental estimate", which is structurally linked to regime.

6.3 Empirical Result

Adding Kalman features improves the MLP's confidence margins without changing the 100% episode accuracy:

	Overall acc	$\bar{p}_{\text{val}}$ | Val	$\bar{p}_{\text{mom}}$ | Mom
MLP (raw)	100%	0.563	0.582
MLP (+Kalman)	100%	0.594	0.611

The improvement in $\bar{p}$ (+0.031 on Val, +0.029 on Mom) means the belief signal fed to PPO is sharper: the policy receives a cleaner discriminative signal between regimes. This should translate to faster policy learning and higher final performance in the E3 experiment.

The LSTM does not benefit from Kalman features (70% in both cases), confirming the bottleneck is data efficiency, not feature quality.

7 Step 6: Full Architecture Overview

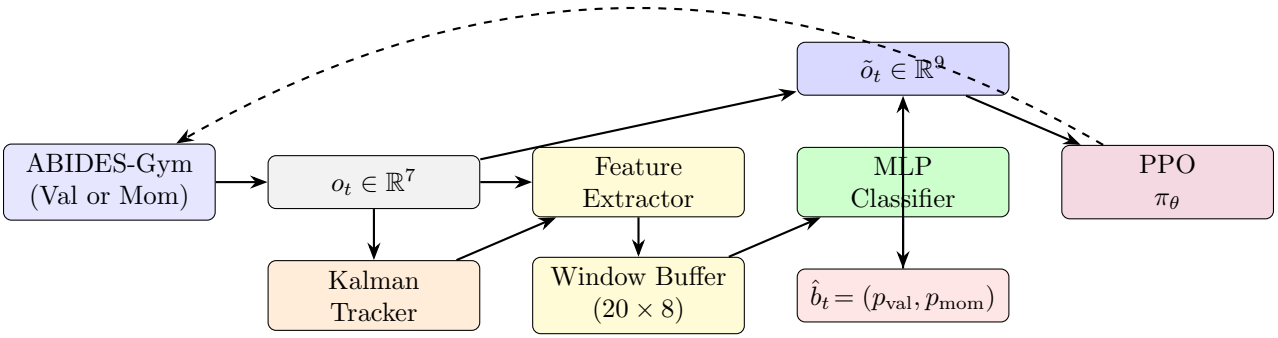

Figure 1: Full pipeline for E3 (trained belief). The Kalman tracker and MLP classifier run at each step alongside the environment. The PPO policy acts on the 9-dim augmented observation.

7.1 RegimeRandomizedGymnasiumEnv

The outer Gymnasium adapter `RegimeRandomizedGymnasiumEnv` handles:

1. **Regime randomisation**: at each `reset()` it samples Val or Mom with probability `val_prob` (default 0.5) and instantiates a fresh ABIDES environment with the corresponding background config.
2. **Belief wrapper**: wraps the base env with the appropriate wrapper (`OracleRegimeWrapper`, `TrainedRegimeBeliefWrapper`, or no wrapper) depending on `belief_mode`.
3. **Gymnasium API**: converts the legacy Gym API (single-value done flag) to Gymnasium's (terminated / truncated split) required by RLlib.
4. **Obs space correctness**: the observation space of the outer env is always set from the *base* env's raw obs dimensions, then $N_{\text{belief}} = 2$ features are added. This avoids double-counting that would arise from probing the already-wrapped inner env.

8 Implementation Notes

8.1 Bug Fixed: Observation Space Double-Counting

The original probe logic for computing the outer env's observation space was:

```
probe = self._make_env("mom")          # returns wrapped env (obs 9x1)
obs_dim = np.prod(probe.obs_space)     # = 9
obs_dim += N_BELIEF_FEATURES           # = 11  -- WRONG
```

For `belief_mode="oracle"`, `_make_env()` returns an `OracleRegimeWrapper` whose obs space is already (9×1). Adding 2 more gives 11, but the actual obs returned by `reset()` is 9 after flattening. This mismatch causes RLlib policy shape errors.

Fix: always probe the base `SubGymMarketsDailyInvestorEnv_v0` with no wrapper, obtain the raw 7-dim obs space, then add 2 for belief:

```
_probe_base = SubGymMarketsDailyInvestorEnv_v0(**_probe_kw)
raw_obs_dim = np.prod(_probe_base.observation_space.shape)  # = 7
obs_dim = raw_obs_dim + N_BELIEF_FEATURES  # = 9  -- correct
```

8.2 Checkpoint Format

Saving only `model.state_dict()` makes loading fragile: the caller must know the architecture independently. All checkpoints now save a full dict:

```
{
  "state_dict":  <OrderedDict>,
  "model_type":  "mlp" | "lstm",
  "use_kalman":  bool,
  "seq_len":     int,
  "n_features":  int,
  "hidden_size": int,
}
```

The `load_classifier(path)` helper reconstructs the correct model class and architecture from this dict, loads the weights, sets eval mode, and returns `(model, model_type, use_kalman, seq_len)`.

8.3 CRITICAL_ENV_BUG: reset() does not clear marked-to-market

ABIDES-Gym does not reset `env.previous_marked_to_market` on `env.reset()`. If not manually corrected, the first reward of a new episode is computed relative to the terminal holdings of the previous episode, producing a large spurious reward signal that destabilises PPO. Every wrapper in this codebase adds:

```
obs = self.env.reset()
self.env.previous_marked_to_market = self.env.starting_cash
```

immediately after `env.reset()`.

9 Experimental Design for E3

9.1 Experiment Variants

Experiment	Belief source	Obs dim	Purpose
E0 (baseline)	None	7	Belief-blind PPO on mixed regimes
E1 (oracle)	Oracle one-hot	9	Upper bound on belief value
E3 (trained)	MLP+Kalman	9	Learned belief, deployable

9.2 Training Protocol

- **Episode composition**: 50% Val, 50% Mom per batch (`val_prob=0.5`).
- **Timestep budget**: 386 steps per episode (one trading day at 60s intervals after 5-min warmup).
- **Reward**: dense marked-to-market P&L, same as v4 baseline.
- **Pass gate**: must beat v4 PPO mean P&L = +32,854 ¢, Sharpe = 2.51, win rate = 100% (evaluated on pure Mom episodes as in v4).

9.3 Hypothesis

The MLP+Kalman classifier achieves 100% episode-level accuracy with $\bar{p} \approx 0.60$ after only 20 training episodes per regime. This means the PPO policy will receive a consistent, correct belief signal within the first few steps of any episode. The hypothesis for E3 is:

> A PPO agent trained with MLP+Kalman regime beliefs will exceed the v4 baseline on mixed-regime test episodes, and will approach (though not reach) E1 oracle performance.

The gap between E3 and E1 is expected to be driven primarily by the soft confidence margins ($\bar{p} \approx 0.60$ vs $p = 1.0$ for oracle), not by episode-level misclassification.

9.4 Ablation Plan

To attribute performance gains, the following ablations should be run in order:

1. E0 (no belief) on mixed Val+Mom: establishes the cost of regime uncertainty without belief.
2. E1 (oracle) on mixed Val+Mom: establishes the value of perfect regime information.
3. E3 (MLP+Kalman) on mixed Val+Mom: the target experiment.
4. E3 (MLP raw, no Kalman) on mixed Val+Mom: ablates the Kalman contribution.
5. E3 with 60-episode retrained classifier: tests whether more training data sharpens belief margins and improves PPO.

10 Saved Artifacts

File	Description
`scripts/regime_classifier.py`	Classifier training + inference code
`scripts/regime_belief_wrapper.py`	Gym wrappers + RegimeRandomizedGymnasiumEnv
`scripts/models/mlp_kalman_regime.pt`	Best model (100% acc, $\bar{p} \approx 0.60$)
`scripts/models/mlp_raw_regime.pt`	MLP without Kalman features
`scripts/models/lstm_kalman_regime.pt`	LSTM+Kalman (reference; underperforms)
`scripts/models/lstm_raw_regime.pt`	LSTM without Kalman (reference)

11 Summary

1. The oracle regime wrapper (E1) establishes the value of regime information by feeding a hard one-hot b^* to the PPO policy.

2. The Kim (1994) filter was the first attempt at a zero-shot inferred belief (E3). It failed because it models the univariate mid-price process, which is OU in both Val and Mom. The distinguishing signal (order-flow autocorrelation from momentum agents) is invisible to it.

3. A supervised classifier trained on a 20-step sliding window of observable market features succeeds where the Kim filter fails. The MLP outperforms the LSTM because the regime signal is encoded in the joint distribution of a fixed window (accessible to MLP) rather than a long-range temporal dependency (where LSTM excels but requires more data).

4. Adding Kalman belief features (`belief_dev`, `belief_std`) improves the MLP's confidence margins from $\bar{p} \approx 0.57$ to $\bar{p} \approx 0.60$, sharpening the signal fed to PPO without changing episode accuracy.

5. The full E3 pipeline (`RegimeRandomizedGymnasiumEnv` with `belief_mode="trained"` and `mlp_kalman_regime.pt`) is ready for PPO training. An obs-space double-counting bug from the probe logic was found and fixed during integration.